Tutorial Django krok po kroku: Prosty System Zgłoszeń

Kompleksowy przewodnik dla początkujących

Przegląd tutoriala

2026

Co zbudujesz: System Zgłoszeń

Opis projektu:
Aplikacja webowa do zarządzania zgłoszeniami

Główne funkcje:

- Dodawanie nowych zgłoszeń
- Wyświetlanie w tabeli
- Edycja istniejących
- Usuwanie z potwierdzeniem

Tech stack:
Python ▪ Django ▪ SQLite ▪ Bootstrap

Operacje CRUD

CRUD

Create

Read

Update

Delete

Dla kogo jest ten tutorial?

Idealny dla

- Programistów Python nowych w webdev
- Studentów uczących się backend
- Każdego chcącego nauki Django

Będziesz potrzebować

- Podstawowa Python
- Umiejętność terminala
- Edytor kodu
- Python 3.x
- 2 godziny czasu

NIE potrzebujesz

- Wcześniejszego Django
- Znajomości baz danych
- Frontend/CSS (używamy Bootstrap)

Podstawowe komponenty Django

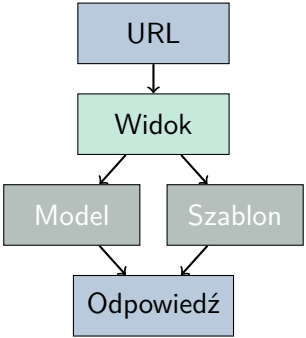
Projekt vs. Aplikacja:

- **Projekt** = kontener całej aplikacji
- **Aplikacja** = moduł z konkretną funkcją

Kluczowe elementy:

Model	struktura danych (tabela)
Widok	logika żądań HTTP
Szablon	HTML z dynamiczną treścią
URL	ścieżki do widoków
Formularz	walidacja danych

Architektura MVT



Twoja ścieżka nauki krok po kroku

Faza 1: Fundamenty (1-6)

- Środowisko
- Projekt i aplikacja
- Model
- Rejestracja aplikacji

Faza 2: Dane (7-9)

- Migracje bazy
- Panel admin
- Rejestracja modelu

Faza 3: UI (10-13)

- Formularze
- Widoki (logika)
- Routing URL
- Szablony HTML

Faza 4: Wykończenie (14-16)

- Stylizacja Bootstrap
- Kompletny CRUD
- Testowanie

Dlaczego ten tutorial się wyróżnia?

Podejście mikro-krokowe

Każde zadanie na najmniejsze kroki. Bez założeń o wiedzy.

Nauka wizualna

Zrzuty ekranu przy każdym kroku. Widzisz co powinno być.

Wieloplatformowe

Polecenia dla Windows, macOS, Linux. Działa wszędzie.

Słowniczek

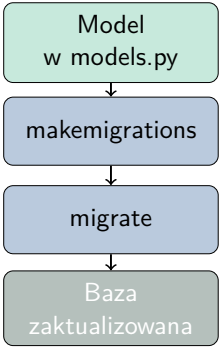
Terminologia Django wyjaśniona z góry.

Praca z danymi: Model i migracje

Model: Zgłoszenie

```
class Zgloszenie(models.Model):
    imie_nazwisko = models.CharField(
        max_length=100,
        verbose_name="Imię i nazwisko"
    )
    email = models.EmailField(
        verbose_name="Adres e-mail"
    )
    temat = models.CharField(
        max_length=150,
        verbose_name="Temat"
    )
    tresc = models.TextField(
        verbose_name="Treść"
    )
    data_utworzenia =
        models.DateTimeField(
            auto_now_add=True
        )
```

Proces migracji:



Obsługa żądań: Widoki i routing URL

Cztery widoki dla CRUD:

Widok	Cel	Metoda
`lista_zgloszen`	Wyświetl wszystkie	GET
`dodaj_zgloszenie`	Utwórz nowe	GET + POST
`edytuj_zgloszenie`	Zaktualizuj	GET + POST
`usun_zgloszenie`	Usuń	GET + POST

Wzorce URL:

```
urlpatterns = [
    path("", views.lista_zgloszen, name="lista_zgloszen"),
    path("dodaj/", views.dodaj_zgloszenie, name="dodaj_zgloszenie"),
    path("edytuj/<int:pk>/", views.edytuj_zgloszenie, name="edytuj_zgloszenie"),
    path("usun/<int:pk>/", views.usun_zgloszenie, name="usun_zgloszenie"),
]
```

Przepływ: URL → Widok → Model → Szablon → Odpowiedź

Interfejs: Szablony i stylizacja

Struktura szablonów:

- `base.html` – layout
- `lista_zgloszen.html` – tabela
- `formularz_zgloszenia.html` – formularz
- `potwierdz_usuniecie.html` – potwierdzenie

Funkcje szablonów:

- `extends` – dziedziczenie
- `block` – placeholdery
- `zmienna` – dane dynamiczne

Integracja Bootstrap:

- Link CDN w `<head>`
- Klasy: `btn`, `card`, `table`
- Zero własnego CSS!

Rezultat

Profesjonalny wygląd
bez pisania CSS!

Wbudowany panel administratora Django

Co dostajesz za darmo:

- Kompletny interfejs admin
- Brak kodu dla CRUD
- Autentykacja
- Auto-generowanie formularzy

Konfiguracja:

1. Rejestruj w `admin.py`
2. `createsuperuser`
3. Dostęp: `/admin/`

Zastosowania:

- CMS
- Narzędzia wewnętrzne
- Administracja danymi
- Prototypowanie MVP

Formularze: Kod formularza zgłoszenia

Plik: `zgloszenia/forms.py`

```
from django import forms
from .models import Zgloszenie

class ZgloszenieForm(forms.ModelForm):
    class Meta:
        model = Zgloszenie
        fields = ["imie_nazwisko", "email", "temat", "tresc"]
        widgets = {
            "imie_nazwisko": forms.TextInput(attrs={"class": "form-control"}),
            "email": forms.EmailInput(attrs={"class": "form-control"}),
            "temat": forms.TextInput(attrs={"class": "form-control"}),
            "tresc": forms.Textarea(attrs={"class": "form-control", "rows": 5}),
        }
```

ModelForm automatycznie generuje formularz z modelu + klasy Bootstrap

Czym są formularze:

- Walidacja danych użytkownika
- Automatyczne renderowanie pól HTML
- Ochrona przed atakami (CSRF)
- Czyszczenie i normalizacja danych

Umiejętności i wiedza praktyczna

Umiejętności techniczne:

- Konfiguracja projektu Django
- Modelowanie bazy danych
- Obsługa formularzy
- Routing URL i widoki
- System szablonów
- Pliki statyczne i CSS

Koncepcje transferowalne:

- Wzorce MVC/MVT
- Myślenie RESTful (CRUD)
- Migracje baz danych
- Cykl żądanie/odpowiedź
- Organizacja kodu

Podsumowanie

Czego się nauczyłeś:

- Tworzenie modeli danych i migracji bazy
- Implementacja pełnego CRUD (Create, Read, Update, Delete)
- Praca z formularzami i walidacją danych
- Routing URL i logika widoków Django
- Renderowanie szablonów z dynamiczną treścią
- Stylizacja interfejsu za pomocą Bootstrap

Następne kroki:

- Rozbuduj aplikację o nowe funkcje
- Zapoznaj się z oficjalnym tutorialem Django
- Eksperymentuj z Django REST Framework